Compiled Injection: Deterministic Activation Engineering for Precise Behavioral Control in Pretrained LLMs

Douglas Rawson
rawson.douglas@gmail.com

May 2026

Abstract

We present *Compiled Injection*—a suite of gradient-free techniques for surgically editing the behavior of pretrained large language models through direct manipulation of activations in the residual stream. Unlike fine-tuning approaches that modify all model parameters and cause catastrophic forgetting, Compiled Injection targets specific layers, positions, and decode steps with precisely computed steering vectors, achieving exact multi-token behavioral outputs while preserving all original model capabilities.

The key innovation is *per-step token injection*, which solves the compound-probability collapse problem—the phenomenon whereby static activation deltas fail to produce multi-token target sequences because a single bias vector must win the argmax at every decode position simultaneously. By swapping the injected delta between forward passes, we collapse the N-token generation problem into N independent single-token problems, reducing failure from exponential to linear in sequence length.

We catalog eight distinct injection mechanisms, including positive/negative anchoring for protocol-level behavioral shifts, composed attention+FFN injection for simultaneous mode and content control, and a systematic concept compilation pipeline using ridge-regularized pseudo-inverse with SVD truncation for computing optimal low-rank weight deltas. We document the full discovery history, including the 8 failed static approaches that motivated the per-step solution.

All mechanisms are fully reversible, auditable, and compatible with quantized models. No gradient descent is required at any stage.

1 Introduction

Large language models are typically modified through gradient-based fine-tuning, which adjusts all model parameters to minimize a loss function. While effective, this approach has fundamental limitations: it causes catastrophic forgetting of prior capabilities, provides no audit trail of which parameters were changed or why, and requires significant computational resources.

We propose an alternative paradigm: *compiled injection*—directly manipulating model activations at specific layers, positions, and decode steps with deterministically computed steering vectors. This approach is analogous to writing firmware rather than retraining an entire operating system: targeted, precise, and reversible.

2 Theory

2.1 The Residual Stream Model

A transformer decoder layer computes:

$$h_{\text{out}} = \text{FFN}(\text{LayerNorm}(h + \text{Attention}(h))) + (h + \text{Attention}(h))$$

At the final position (the last token), the hidden state h encodes everything the model knows about the sequence. The `lm_head` projects this state to vocabulary logits:

$$\text{logits} = W_{\text{lm}} \cdot \text{LayerNorm}(h[-1, :])$$

$$\text{next\_token} = \arg\max(\text{logits})$$

All token predictions flow through that single vector at the last position. Modifying it modifies what the model predicts next.

2.2 Unembedding Rows as Steering Vectors

The `lm_head` weight matrix $W_{\text{lm}} \in \mathbb{R}^{V \times d}$ has one row per vocabulary token. Each row $W_{\text{lm}}[t, :]$ is the *unembedding vector* for token t. When h aligns with $W_{\text{lm}}[t, :]$, the logit for t is maximized.

If we add a scaled copy of $W_{\text{lm}}[t, :]$ to the hidden state:

$$h' = h + \alpha \cdot \frac{W_{\text{lm}}[t, :]}{\|W_{\text{lm}}[t, :]\|}$$

Then the logit for token t receives an additional term proportional to $\alpha \cdot \|W_{\text{lm}}[t, :]\|$, strongly biasing the model toward predicting t. This is the mathematical justification for using unembedding rows as steering vectors.

2.3 The Compound-Probability Collapse

Consider injecting a static delta vector v at every decode step. For a target sequence of N tokens $(t_1, \ldots, t_N)$, the probability of correctly generating the full sequence is:

$$P_{\text{correct}} = \prod_{k=1}^{N} P(t_k \mid \text{context}_{<k},\ + \text{static delta})$$

Each factor $P(t_k \mid \ldots)$ is strictly < 1 for any injection weaker than complete dominance. The product decays *exponentially* with sequence length N. Even if each token has 90% probability with a well-tuned static delta, a 10-token sequence has only $0.9^{10} \approx 35\%$ chance of complete success.

Empirically, this manifests as one of two failure modes: at low alpha, the model ignores the injection entirely; at high alpha, the same injected token wins every position (e.g., `getgetgetgetget`...), because the unchanging delta continues to win the argmax for that same token.

2.4 Per-Step Injection: The Solution

Per-step injection replaces the compound product with N independent single-token problems. At each decode step k, a different steering vector $v_k = \frac{W_{\text{lm}}[t_k, :]}{\|W_{\text{lm}}[t_k, :]\|}$ is injected—and then *swapped out* before the next forward pass. This prevents the previously-injected token from re-winning.

The probability of success becomes:

$$P_{\text{correct}} = \min_{k=1}^{N} P(t_k \mid \text{context}_{<k},\ + \text{delta}_k)$$

This is a *linear* degradation in sequence length (bottlenecked by the hardest single token), rather than exponential. Empirically, this change alone enables exact multi-token sequence injection where all 8 static approaches failed.

2.5 Attention vs. FFN: Two Axes of Control

The attention mechanism and the feed-forward network serve different roles:

- **Attention injection**: Sets the model's "protocol class"—the mode or style of the response (e.g., JSON vs. prose, English vs. French). Static deltas suffice because the protocol should remain constant throughout generation.
- **FFN injection**: Provides per-position token precision. Requires per-step delta swapping because different tokens are needed at different decode positions.

Combined, they cover both dimensions of behavioral control.

3 Mechanism Catalog

3.1 1. Per-Step Token Injection

Purpose: Make the model output an exact multi-token sequence.

Method: At decode step k, inject steering vector v_k at the last position of a target layer's activations. Swap to v_{k+1} before the next forward pass. Clear after target sequence length.

Parameters: Target layer (14–23), alpha (100–500 for 0.5B model, scale proportionally for larger models), trigger condition for selective activation.

3.2 2. Positive/Negative Anchoring

Purpose: Shift the model's behavioral protocol—not specific tokens, but the class of output.

Method: Static attention deltas at specific layers. Positive anchors add $\alpha \cdot v_{\text{pos}}$ pulling toward desired behavior; negative anchors subtract $\alpha \cdot v_{\text{neg}}$ pushing away from undesired patterns. Combined, they provide complete protocol-level control.

Empirical Result: JSON formatting proof—combined positive anchoring (L14+L20, $\alpha = 0.3 - 0.5$) and negative anchoring (L20+L23, $\alpha = 2.0$) produced clean JSON output across all prompt types, including previously resistant variations.

3.3 3. Text Injection (CodingEngine Pattern)

Purpose: Interrupt the model mid-generation and inject corrective text.

Method: Sliding-window detector identifies trigger patterns in generated tokens. When triggered, append corrective text to the context (not hidden state manipulation). The injected text ends with the same pattern that triggered it, enabling seamless generation resumption. A `fired_rules` set prevents re-triggering.

Empirical Result: Redirected a model from writing an incorrect bug fix (`_separable`) to the correct fix (`_cstack, =1→=right`) by detecting the diff format trigger and injecting methodology guidance.

3.4 4. Composed Injection

Purpose: Combine protocol-level and token-level control.

Method: Install both Attention injection (static, sets mode) and FFN injection (per-step, provides tokens) simultaneously. Co-tune alphas: reduce attention alpha by $4\times$ in the composed regime to prevent one mechanism from dominating the other.

3.5 5. Logit Biasing

Purpose: Simple token-level control via logit vector manipulation.

Method: Boost or suppress specific token IDs in the output logits before sampling. Per-step biasing avoids the compound-probability collapse. Effective for safety filtering (suppress dangerous tokens: `eval`, `exec`, `pickle`) and guaranteed single-token output.

3.6 6. Compiled FFN Rules

Purpose: Permanent weight-level trigger → response rules via FFN dimension expansion.

Method: Add new intermediate dimensions to FFN layers implementing deterministic lookup rules. Each rule is a rank-1 FFN addition: $\text{ReLU}(x \cdot \text{gate}_k - \theta_k) \cdot \text{value}_k$. Empirically holds 50,000+ rules in d=24 dimensions with zero cross-talk.

3.7 7. Concept Compilation Pipeline

Purpose: Systematic delta computation from multiple (p, r) examples.

Method: Capture FFN input residuals for prompt-only and prompt+target runs. Solve for optimal low-rank weight delta via ridge-regularized pseudo-inverse $\Delta W = A^{\dagger} \cdot (B - A)$ followed by SVD truncation to rank k. Apply via reversible wrapper. Verify with parity testing. Rollback guaranteed by context manager.

3.8 8. The Plugin System

Purpose: Stackable, reversible hooks for combining multiple injection mechanisms.

Method: Each plugin implements hooks for `pre_generate`, `on_logits`, and `post_generate`. Plugins are ordered (later sees earlier's output), composable, and reversible. Concrete implementations include SafetyPlugin (token suppression), ConceptPatchPlugin (FFN delta application), and APIRulePlugin (cosine-matched trigger → answer logit biasing).

4 History: Discovery Timeline

The development of Compiled Injection proceeded through several stages during May 2026:

4.1 Eight Failed Approaches

The initial attempt used a single static steering vector, applied identically at every decode step. Eight variations were tested: static deltas at the FFN, attention, lm_head, and embedding levels; prompt injection; global logit biasing; multi-layer simultaneous injection; and per-token weighted deltas. All eight failed for multi-token targets, consistently producing either no effect (low alpha) or the same token at every position (high alpha).

4.2 The Breakthrough: Per-Step Injection

The hypothesis: making the delta a function of the decode step would break the compound-probability collapse. A stateful wrapper (`StepwiseMLP`) was created that could swap its injected delta between forward passes. Testing against a 4-token target sequence showed exact matches at multiple layer/alpha combinations where all static approaches failed. Scale validation confirmed the mechanism across 1.5B fp16 and 7B 4-bit models.

4.3 Attention Injection and Composition

Extending the wrapper pattern to attention layers revealed that static attention deltas are sufficient for protocol-level shifts (language, formatting, mode)—attention controls branch selection rather than per-token emission. Composing both mechanisms required alpha co-tuning: attention alpha reduced by 4× in the composed regime.

4.4 Concept Compilation and Systematic Methods

The approach was generalized from hand-tuned steering vectors to systematic delta computation via pseudo-inverse + SVD. This enabled rank-k FFN patches that change output behavior, positive/negative anchoring for complete behavioral control, and the CodingEngine pattern for mid-generation text injection with sliding-window trigger detection.

4.5 Persistence and the Plugin System

The final stages generalized the wrapper pattern into a stackable, reversible plugin architecture and extended it to persistent storage: compiled deltas survive model save/load roundtrips via persistent buffers on the model root.

5 Conclusion

Compiled Injection demonstrates that complex behavioral modifications of pretrained language models can be achieved deterministically, without gradient descent, through targeted activation-space manipulation. The key insight—that per-step delta swapping solves the compound-probability collapse—enables exact multi-token behavioral control while preserving all original model capabilities.

These techniques represent a new paradigm for model control: knowledge and behaviors compiled into the forward pass rather than trained through optimization. They are fully auditable, reversible, and compatible with existing deployment infrastructure.

This document establishes prior work on Compiled Injection techniques as of May 16, 2026.
Repository: https://github.com/DRawson5570/compiled-injection
Contact: rawson.douglas@gmail.com